

Cleft - เอกสารสรุปโปรเจกต์ฉบับ Portfolio

อัปเดตล่าสุด: 3 กันยายน 2026

สถานะ: โค้ดบน `main` ผ่าน GitHub Actions ทั้ง Backend, Frontend, browser tests และ design prototype

จุดประสงค์: ใช้ทำความเข้าใจระบบ เตรียมอธิบายผลงาน และเป็น checklist ก่อน deploy

1. Executive Summary

Cleft คือ Full Stack web application สำหรับแบ่งค่าใช้จ่ายของกลุ่ม ผู้ใช้สร้างบิล เพิ่มรายการและจำนวน เพิ่มสมาชิก กำหนดว่าสมาชิกแต่ละคนร่วมจ่ายทุกรายการหรือเฉพาะรายการ ตรวจสอบยอดที่คำนวณแบบรายสตางค์ แบนสลิป และยืนยันบิลลง PostgreSQL เพื่อดูประวัติภายหลัง

โปรเจกต์นี้แสดงความสามารถที่เหมาะสมกับ portfolio ด้านต่อไปนี้:

- ออกแบบระบบแยก Frontend, Backend และ Database อย่างชัดเจน
- สร้าง authentication ด้วย Google OAuth, JWT และ HTTP-only cookies
- ออกแบบ business rule สำหรับการหารเงินที่ตรวจสอบผลรวมได้
- จัดการข้อมูลสัมพันธ์ด้วย Prisma migration และ transaction
- ทำ responsive UI และมีหน้า demo ที่ไม่ต้องล็อกอิน
- ทำ automated quality gates ตั้งแต่ lint ถึง browser test
- เตรียม Docker Compose และ environment validation สำหรับการนำระบบไปรัน

สิ่งที่ Cleft ไม่ได้อ้างว่าเป็นในสถานะปัจจุบัน:

- ไม่ใช่ระบบโอนเงินจริง
- ยังไม่ใช่ระบบ settlement หรือระบบติดตามหนี้ครบวงจร
- ยังไม่ได้ยืนยัน production deployment ด้วย credentials จริง
- design prototype ใน `edit_front_end/` ไม่ใช่ production frontend

2. Problem และแนวคิดของผลิตภัณฑ์

การหารบิลด้วยมือมักผิดพลาดเมื่อสมาชิกแต่ละคนกินไม่เหมือนกัน มีหลายรายการ หรือราคาอาหารไม่ลงตัว Cleft แก้ปัญหานี้ด้วยการแยกข้อมูลเป็น item และ participant แล้วคำนวณผู้รับผิดชอบของแต่ละ item โดยตรง

เส้นทางหลักของผู้ใช้:

1. Login ด้วย Google

2. เพิ่มชื่อปาร์ตี้ วันที่ รายการ ราคา จำนวน และหมายเหตุ
3. เพิ่มสมาชิกและเลือก ALL หรือ PARTIAL
4. ตรวจสอบรายการที่ยังไม่มีผู้รับผิดชอบ
5. ดูยอดต่อคนและยอดรวมใน Summary
6. แบนรูปสลิปถ้าต้องการ
7. Confirm เพื่อบันทึกข้อมูลแบบ transaction
8. ดูบิลล่าสุดจากหน้า Home

สำหรับ reviewer ที่ไม่ต้องการตั้งค่า OAuth มีหน้า /demo แสดงตัวอย่างบิลและผลการแบ่งโดยไม่อ่านหรือเขียนฐานข้อมูล

3. ขอบเขตความสามารถปัจจุบัน

3.1 Authentication และผู้ใช้

- เริ่ม Google OAuth ผ่าน /api/auth/google
- รับ callback ผ่านโดเมน Frontend ที่ /api/auth/google/callback
- ใช้ access token และ refresh token ใน HTTP-only cookies
- หมุน refresh token เมื่อเรียก refresh endpoint
- เก็บเฉพาะ SHA-256 hash ของ refresh token ในฐานข้อมูล
- Logout พร้อม revoke refresh token ที่บันทึกไว้
- อ่านและแก้ username, avatar และ currency symbol

3.2 การสร้างและแบ่งบิล

- เพิ่ม แก้ไข และลบ item
- รองรับราคาต่อชิ้น quantity และ note
- ป้องกันชื่อ item ซ้ำโดยไม่สนตัวพิมพ์ใหญ่เล็ก
- เพิ่ม แก้ไข และลบ member
- ป้องกันชื่อ member ซ้ำ
- เปลี่ยนชื่อหรือลบ item พร้อมปรับ reference ของสมาชิก PARTIAL
- แบ่งแบบ ALL และ PARTIAL
- ป้องกัน Confirm ถ้ายังมี item ที่ไม่มีสมาชิกคนใดรับผิดชอบ
- กระจายเศษสตางค์แบบ deterministic เพื่อให้ยอดสมาชิกรวมเท่ากับยอดบิล
- แบนรูป JPG, JPEG, PNG, GIF หรือ WebP ขนาดไม่เกิน 5 MB
- บันทึก Party, Item, Participant, Consumption และ History ภายใน transaction เดียว

3.3 ประสิทธิภาพใช้งาน

- Draft ถูกเก็บใน localStorage ภายใต้ key cleft-session

- หน้า Home แสดงบิลปัจจุบัน ประวัติล่าสุด และกราฟการกระจายยอด
- หน้า Summary แสดงยอดรายคน stacked bar และ digital slip
- หน้า `/demo` ใช้ข้อมูลคงที่สำหรับนำเสนอ portfolio
- UI รองรับ desktop และ mobile
- มี loading state, toast และ auth guard

4. Technology Stack

ชั้นระบบ	เทคโนโลยี	หน้าที่หลัก
Production Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	Routes, UI, local draft และ API client
Client Data	TanStack Query, Axios	Request cache, profile/history query และ refresh flow
Visualization	Recharts	Pie chart และสัดส่วนค่าใช้จ่าย
Production Backend	NestJS 12, TypeScript	REST API, validation, authentication และ business logic
Authentication	Passport, Google OAuth, JWT	Login, session, refresh และ logout
Database Access	Prisma 5	Schema, migration, query และ transaction
Database	PostgreSQL 16	จัดเก็บผู้ใช้ บิล สมาชิก รายการ และประวัติ
Upload	Multer	จำกัดขนาด ตรวจสอบชนิด และเขียนไฟล์สลิป
Test	Vitest, Testing Library, Playwright	Unit, component, browser และ integration tests
Delivery	Docker, Docker Compose, GitHub Actions	Local stack และ automated CI
Design Archive	Vite, React	เก็บ UI prototype เพื่อแสดงพัฒนาการของงานออกแบบ

5. Repository Map

```
Project_github/
|-- frontend/           Production Next.js application
|   |-- app/            Home, Items, Members, Summary, Demo
|   |-- components/     Navigation, charts และ UI primitives
|   |-- hooks/          Party draft state และ localStorage
|   |-- lib/            API client, types และ bill calculation
|   |-- e2e/            Playwright browser tests
|   |-- test/           Vitest component/helper tests
|   `-- public/         Cleft icons และ PWA assets
|-- backend/            Production NestJS API
|   |-- src/auth/       OAuth, JWT, refresh และ logout
|   |-- src/users/      Profile API
|   |-- src/party/      Calculate, confirm และ history
|   |-- src/prisma/     Database service
|   |-- src/config/     Environment validation
|   |-- prisma/         Schema และ committed migrations
|   `-- scripts/        API integration smoke test
|-- edit_front_end/     Archived design prototype
|-- uploads/            Local/Docker slip storage
|-- scripts/            Project utilities
|-- .github/workflows/ci.yml Automated quality pipeline
|-- docker-compose.yml  Full local stack
|-- PORTFOLIO.md        ทางลัดสำหรับ reviewer
|-- SUMMARY.md           Source ของเอกสารนี้
`-- output/pdf/SUMMARY.pdf PDF สำหรับอ่านและส่งประกอบ portfolio
```

ไฟล์ `.env` จริง, build output, test report และไฟล์ upload ถูก ignore จาก Git เฉพาะ `.env.example` และ `uploads/.gitkeep` เท่านั้นที่ควรอยู่ใน repository

6. System Architecture

```
Browser
|
| pages และ same-origin /api requests
v
Next.js Frontend :3000
|-- React UI
|-- TanStack Query และ Axios
|-- localStorage draft
|-- /demo แบบไม่ใช้บัญชี
|
| rewrite /api/:path*
v
NestJS Backend :4000
|-- Helmet, CORS, cookie parser
|-- Passport JWT guard
|-- DTO validation
|-- Auth, User และ Party modules
|-- Multer slip upload
v
Prisma ORM
|-- committed migration
|-- transaction ตอน Confirm
v
PostgreSQL :5432
```

Frontend ใช้ path `/api` เสมอ Next.js rewrite จะส่ง request ต่อไปยัง Backend ตาม `API_PROXY_TARGET` วิธีนี้ทำให้ client ไม่ต้องรู้ URL ภายในของ Backend และช่วยให้ cookies ถูกใช้งานใน origin เดียวกับหน้าเว็บ

7. Data Flow สำคัญ

7.1 Google Login

```
User clicks Google login
-> GET /api/auth/google
-> Next.js rewrite
-> NestJS Google guard
-> Google consent
-> /api/auth/google/callback ผ่าน Frontend domain
-> User upsert ใน PostgreSQL
-> access/refresh tokens
-> HTTP-only cookies
-> redirect กลับ Frontend
```

Callback ผ่าน Frontend URL เพื่อให้ response ที่ตั้ง cookie กลับมาบน origin ที่ผู้ใช้เปิดอยู่ การตั้งค่า `GOOGLE_CALLBACK_URL` ใน Google Cloud ต้องตรงกับค่าจริงทุกตัวอักษร

7.2 Draft Bill

Items/Members forms

- > React state
- > usePartySession
- > localStorage: cleft-session
- > bill-display helpers
- > Summary preview

Draft ยังไม่ถูกส่งเข้า Database จนกว่าจะ Confirm จึงแก้ไขได้เร็วและไม่สร้างข้อมูลค้างใน server แต่ draft จะอยู่เฉพาะ browser และอุปกรณ์นั้น

7.3 Confirm Bill

Summary validation

- > normalize quantity เป็น line total
- > multipart FormData: JSON payload + optional slip
- > JWT guard
- > JSON parse และ nested DTO validation
- > business-rule validation
- > Prisma transaction
- > Party + Items + Participants + Consumptions + History
- > clear local draft

ถ้าการเขียนฐานข้อมูลส่วนใดล้มเหลว Prisma transaction จะ rollback record ในฐานข้อมูลทั้งหมด

8. Bill Calculation และ Data Integrity

8.1 Split Rules

- `ALL` : สมาชิกคนนั้นถูกเพิ่มเป็น assignee ของทุก item
- `PARTIAL` : สมาชิกคนนั้นเป็น assignee เฉพาะชื่อ item ที่อยู่ใน `itemNames`
- `PARTIAL` ที่ไม่เลือก item มียอด 0 แต่ระบบจะไม่อนุญาตให้มี item ใดไม่มี assignee เลย

8.2 วิธีคำนวณ

Frontend แปลงราคาต่อชิ้นเป็น line total ก่อนส่ง Backend:

```
lineTotal = unitPrice x quantity
```

สำหรับแต่ละ item ระบบทำงานเป็นจำนวนเต็มหน่วยสตางค์:

```
itemCents = round(lineTotal x 100)
baseShare = floor(itemCents / assigneeCount)
remainder = itemCents mod assigneeCount
```

ทุกคนได้รับ `baseShare` และ `remainder` ถูกแจกทีละ 1 สตางค์ตามลำดับ `assignee` ที่คงที่ ผลรวมยอดรายคนจึงเท่ากับยอดบิลเสมอ

ตัวอย่าง 100 บาท หาร 3 คน:

คน	ยอด
Alice	33.34
Bob	33.33
Cara	33.33
รวม	100.00

8.3 Validation ที่ป้องกันข้อมูลผิด

- Party ต้องมีชื่อและวันที่ที่ parse ได้
- ต้องมีอย่างน้อย 1 item และ 1 participant
- ราคา item ต้องเป็นจำนวนบวก
- ชื่อ item และ participant ต้องไม่ว่างและไม่ซ้ำหลัง trim/case normalization
- `PARTIAL.itemNames` ต้องอ้างถึง item ที่มีอยู่จริง
- ทุก item ต้องมี assignee อย่างน้อยหนึ่งคน
- Backend คำนวณและ validate เอง ไม่เชื่อผลที่คำนวณจาก browser

9. REST API

Method	Endpoint	Auth	หน้าที่
GET	<code>/health</code>	ไม่ต้อง	Backend health check
GET	<code>/auth/google</code>	ไม่ต้อง	เริ่ม Google OAuth
GET	<code>/auth/google/callback</code>	Google guard	รับ OAuth result และตั้ง cookies
GET	<code>/auth/session</code>	JWT	อ่าน session ปัจจุบัน

Method	Endpoint	Auth	หน้าที่
POST	/auth/refresh	Refresh cookie	ตรวจและหมุน refresh token
POST	/auth/logout	JWT	Revoke token และล้าง cookies
GET	/users/profile	JWT	อ่าน profile
PATCH	/users/profile	JWT	แก้ profile
POST	/party/calculate	JWT	Validate และคำนวณบิล
POST	/party/confirm	JWT	บันทึกบิลและ optional slip
GET	/party/history	JWT	ประวัติล่าสุด 3 รายการ
GET	/party/history?all=true	JWT	ประวัติทั้งหมด

เมื่อ request ได้ 401 Axios interceptor จะพยายาม refresh หนึ่งครั้ง และ request ที่เกิดพร้อมกันจะรอ refresh promise เดียวกัน เพื่อลดการส่ง refresh ซ้ำ

10. Database Model

```

User 1 -----< History >----- 1 Party
                        |
                        +-----< Item
                        |
                        +-----< Participant
                                |
                                Item >-----< Consumption
  
```

User

เก็บ Google identity, email, username, avatar, currency symbol และ refresh token hash โดย googleId และ email เป็น unique

Party

เก็บชื่อ วันที่ ยอดรวม URL ของสลิป และเวลา created/updated

Item

เก็บชื่อ line total และ note หลัง Confirm ปัจจุบัน quantity ถูกแปลงรวมใน price และบันทึกเป็นข้อความใน note

Participant

เก็บชื่อและ split type ของสมาชิกใน Party

Consumption

เป็น many-to-many join ระหว่าง Participant และ Item ใช้เก็บ item selection ของสมาชิก PARTIAL ด้วย composite primary key

History

เชื่อม User กับ Party ที่ผู้ใช้นั้นสร้าง ใช้เป็น ownership path สำหรับหน้า history

Relations หลักใช้ onDelete: Cascade เพื่อลบ record ลูกเมื่อ parent ถูกลบ

11. Authentication และ Security Posture

มาตรการที่มีแล้ว:

- HTTP-only cookies ลดการอ่าน token จาก JavaScript
- Secure cookie ใน production
- SameSite ตั้งค่าได้เป็น lax หรือ none
- Refresh token ถูก hash ก่อนเก็บ Database
- Refresh token rotation และ revoke ตอน logout
- Helmet security headers
- CORS จำกัดตาม FRONTEND_URL
- Global ValidationPipe ใช้ whitelist และ transform
- Confirm DTO ใช้ forbidNonWhitelisted
- JWT guard ครอบ profile และ party endpoints
- จำกัด upload 5 MB และตรวจ MIME prefix กับ extension
- Production validation บังคับ JWT secret ยาวอย่างน้อย 32 ตัวอักษร
- Secrets จริงไม่อยู่ใน Git

สิ่งที่ต้อง harden ก่อนเปิด public production:

1. เพิ่ม rate limiting ให้ auth, refresh, calculate, confirm และ upload
2. ประเมิน CSRF protection เพิ่มเติม โดยเฉพาะ deployment ที่ต้องใช้ SameSite=None
3. ตรวจ file signature หรือ magic bytes แทนการเชื่อ MIME/extension อย่างเดียว
4. ย้าย slip ไป private object storage และใช้ signed URL เมื่อเหมาะสม
5. ลบไฟล์ที่ Multer เขียนแล้ว หาก payload validation หรือ transaction ล้มเหลว
6. เพิ่ม log, error tracking และ security monitoring ที่ไม่บันทึก token/secret
7. วางนโยบาย backup, restore และ credential rotation

12. Automated Testing และ CI

GitHub Actions ทำงานทุก push ไป `main` และทุก pull request แบ่งเป็น 3 jobs:

12.1 Backend quality and integration

- `npm ci`
- Prisma Client generation
- ESLint
- TypeScript typecheck
- Vitest unit tests
- NestJS production build
- PostgreSQL 16 service
- `prisma migrate deploy`
- API integration smoke test

Backend unit tests ปัจจุบัน 4 เคส ครอบคลุม:

- การแบ่ง `ALL` และ `PARTIAL`
- การแจกเศษสตางค์แบบ deterministic
- การปฏิเสธ item ที่ไม่มี assignee
- การปฏิเสธข้อผิดพลาด normalization

Integration smoke test ครอบคลุม health, authenticated session, profile update, calculation edge cases, confirm, history และ cleanup ข้อมูลทดสอบ

12.2 Frontend quality and browser tests

- `npm ci`
- ESLint
- TypeScript typecheck
- Vitest unit/component tests
- Next.js production build
- Playwright Chromium installation
- Playwright E2E

Frontend unit/component tests ปัจจุบัน 5 เคส ครอบคลุม calculation/display helpers และ shared Button component

Browser tests ปัจจุบัน 3 เคส:

- Signed-out login page แสดง action ที่ถูกต้อง

- Reviewer เปิด `/demo` ได้โดยไม่ต้อง authenticate
- Protected route redirect ผู้ใช้ที่ยังไม่ login

12.3 Archived design prototype

- `npm ci`
- Vite production build
- Vendor code splitting เพื่อลด bundle หลัก

สถานะ CI ล่าสุด ณ วันที่เอกสารนี้ถูกสร้าง: ผ่านทั้ง 3 jobs และ Playwright ผ่าน 3 tests

13. Local และ Docker Runbook

Docker Compose

1. สร้าง `backend/.env` จากตัวอย่างและใส่ค่าจริงสำหรับ local
2. ตรวจสอบ Google callback ให้เป็น `http://localhost:3000/api/auth/google/callback`
3. รัน:

```
docker compose up -d --build
docker compose ps
```

ตรวจสอบระบบ:

```
Frontend: http://localhost:3000
Demo:     http://localhost:3000/demo
Health:   http://localhost:3000/api/health
Backend:  http://localhost:4000/health
```

ลำดับ startup:

```
PostgreSQL healthy
-> Backend runs prisma migrate deploy
-> Backend healthcheck passes
-> Frontend starts
```

Native quality checks

```
cd backend
npm ci
npm run prisma:generate
npm run lint
npm run typecheck
npm run test:unit
npm run build

cd ../frontend
npm ci
npm run lint
npm run typecheck
npm run test:unit
npm run build
npm run test:e2e

cd ../edit_front_end
npm ci
npm run build
```

14. Environment Variables

Backend

Variable	ความหมาย
DATABASE_URL	PostgreSQL connection string
PORT	Backend port ค่าเริ่มต้น 4000
FRONTEND_URL	Allowed origin และ redirect หลัง login
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret
GOOGLE_CALLBACK_URL	Callback ที่ลงทะเบียนใน Google Cloud
JWT_ACCESS_SECRET	Signing secret ของ access token
JWT_REFRESH_SECRET	Signing secret ของ refresh token
JWT_ACCESS_TTL	อายุ access token ค่าเริ่มต้น 15m

Variable	ความหมาย
JWT_REFRESH_TTL	อายุ refresh token ค่าเริ่มต้น 7d
COOKIE_SAME_SITE	lax หรือ none

Frontend

Variable	ความหมาย
API_PROXY_TARGET	Backend URL สำหรับ Next.js rewrite

API_PROXY_TARGET ถูกอ่านตอน Frontend build หากเปลี่ยน Backend URL ต้อง rebuild Frontend image

15. จุดที่ผัดบ่อยและวิธีวิเคราะห์

Backend build ผ่านแต่ start ไม่ได้

ตรวจว่า feature module ที่ใช้ JWT guard ได้ import Passport configuration และ environment variables ครบ

Docker แจ้งว่าไม่พบ backend/.env

สร้างไฟล์จาก backend/.env.example ก่อน Compose startup และอย่า commit ไฟล์จริง

Database เชื่อมต่อได้แต่ไม่มีตาราง

ใช้ committed migration และ prisma migrate deploy ห้ามพึ่ง prisma db push ใน production

Frontend container ติดต่อ localhost:4000 ไม่ได้

ภายใน container localhost หมายถึง container นั้นเอง ค่า build ของ Frontend ต้องใช้ http://backend:4000

เปลี่ยน API URL แล้ว rewrite ยังเป็นค่าเดิม

Rebuild Frontend เพราะ Next.js สร้าง rewrite จาก environment ระหว่าง build

OAuth login สำเร็จแต่ Frontend ไม่มี cookie

ตรวจ GOOGLE_CALLBACK_URL , FRONTEND_URL , HTTPS, Secure และ SameSite ให้สัมพันธ์กัน Callback ควรผ่าน Frontend origin ตามสถาปัตยกรรมปัจจุบัน

ยอดสมาชิกไม่เท่ากับยอดบิล

คำนวณด้วย integer cents ต่อ item และแจก remainder ห้ามหารเป็น floating point แล้วปัดแยกทีหลัง

Confirm ถูกปิดใช้งาน

ตรวจ party name, items, members และข้อความเตือน unassigned item บนหน้า Summary

Build prototype มี bundle warning

ตรวจ `edit_front_end/vite.config.ts` และรักษา vendor code splitting ของ React กับ chart library

16. Gap Analysis - สิ่งที่ยังขาด

ไม่มี blocker ที่ทำให้ source ปัจจุบัน build หรือ CI ไม่ผ่าน แต่ก่อนใช้เป็น production จึงควรจัดการหัวข้อต่อไปนี้

P0 - ต้องทำก่อนเปิดใช้งานสาธารณะ

เรื่อง	สถานะปัจจุบัน	งานที่ควรทำ
Production credentials	ใช้ตัวอย่างใน repo	ตั้ง Google OAuth, Database และ JWT secrets ผ่าน secret manager
Persistent slip storage	เขียนลง local disk	ใช้ S3-compatible/object storage และ cleanup เมื่อ transaction fail
Abuse protection	ยังไม่มี rate limiting	จำกัด request ตาม IP/user โดยเฉพาะ auth และ upload
CSRF review	พึ่ง SameSite/CORS	เพิ่ม CSRF token หรือ origin enforcement หากใช้ cross-site cookies
Operations	ยังไม่มี monitoring/backup runbook	เพิ่ม structured logs, error tracking, uptime check และ restore test

P1 - เพิ่มความน่าเชื่อถือของ portfolio

เรื่อง	ช่องว่าง	งานที่ควรทำ
Real authenticated E2E	Browser tests mock unauthenticated API	เพิ่ม test account หรือ OAuth test strategy สำหรับ full create-confirm-history flow

เรื่อง	ช่องว่าง	งานที่ควรทำ
Auth coverage	Smoke test ยังไม่ครอบคลุม refresh/logout edge cases	เพิ่ม integration tests สำหรับ rotation, revoke, expired และ reused token
Accessibility	ยังไม่มี automated audit	เพิ่ม axe และ keyboard/focus test
API contract	ยังไม่มี OpenAPI page	เพิ่ม Swagger/OpenAPI พร้อมตัวอย่าง error responses
History scaling	<code>all=true</code> โหลดทั้งหมด	เพิ่ม cursor pagination และ detail endpoint

P2 - Product evolution

- เพิ่ม payer model เพื่อระบุว่าใครสำรองจ่าย
- เพิ่ม paid/unpaid และ settlement history
- เก็บ quantity เป็น database field แทนการรวมใน price/note
- เพิ่ม tax, service charge, discount และ tip
- แชรส์บิลด้วย URL หรือ QR code
- Export ใบสรุปเป็นภาพหรือ PDF
- Sync draft ข้ามอุปกรณ์
- รองรับหลายภาษาและหลายสกุลเงินในบิลเดียว
- เพิ่มการแก้ไขหรือลบบิลที่ Confirm แล้วตาม authorization policy

Technical debt ที่ควรรู้

- อายุ cookie ใน controller ถูกกำหนดเป็น 15 นาทีและ 7 วันแยกจาก `JWT_ACCESS_TTL` และ `JWT_REFRESH_TTL` หากแก้ TTL ควรทำให้ cookie max-age ใช้ config เดียวกัน
- Refresh token hash เป็นค่าเดียวต่อ user การ login จากอุปกรณ์ใหม่อาจแทน session ของอุปกรณ์เดิม หากต้องรองรับหลายอุปกรณ์ควรมี Session model
- `confirmedAt` ถูก validate แต่ยังไม่ได้เก็บเป็น field แยก โดยระบบใช้ `createdAt` ของ History เป็นเวลาอ้างอิง
- Upload filter ตรวจสอบ MIME ที่ client ส่งและ extension แต่ยังไม่ตรวจสอบ file signature
- History API ยังไม่คืน consumption detail จึงยังใช้เปิดดู breakdown ของบิลเก่าแบบเต็มไม่ได้

17. Deployment Checklist

ก่อน deploy:

- [] CI ของ commit ที่จะ deploy ผ่านทั้งหมด
- [] ตั้ง production PostgreSQL และทดสอบ connection จาก runtime

- [] ตั้ง Google OAuth Authorized redirect URI ตรงกับ production callback
- [] ตั้ง JWT secrets แบบสุ่มอย่างน้อย 32 ตัวอักษร
- [] ตั้ง FRONTEND_URL , GOOGLE_CALLBACK_URL และ API_PROXY_TARGET ให้ตรงโดเมนจริง
- [] ใช้ HTTPS และตรวจ cookie attributes ผ่าน browser devtools
- [] รัน prisma migrate deploy
- [] ย้าย slip upload ไป persistent storage หรือยืนยันว่า volume ไม่หาย
- [] ทดสอบ login, refresh, logout, create, confirm และ history ด้วยบัญชีจริง
- [] ตรวจ mobile viewport และ keyboard navigation
- [] ตั้ง health monitoring และ database backup
- [] ห้ามนำ .env หรือ credentials เข้า Git

หลัง deploy:

- [] /health ตอบ 200
- [] /api/health ผ่าน Frontend proxy ตอบ 200
- [] OAuth callback กลับหน้า Frontend และ cookies ถูกตั้งครบ
- [] Confirm บิลสำเร็จและข้อมูลปรากฏใน history
- [] Restart service แล้วประวัติและไฟล์สลิปยังอยู่
- [] ตรวจ log ว่าไม่มี token, secret หรือ personal data ที่ไม่จำเป็น

18. Interview Walkthrough

ลำดับนำเสนอประมาณ 5 ถึง 7 นาที:

1. เปิด /demo อธิบายปัญหาและ UI ภายใน 30 วินาที
2. เปิด Items และ Members อธิบาย ALL กับ PARTIAL
3. เปิด calculation helper และ Backend PartyService อธิบาย integer cents กับ deterministic remainder
4. เปิด Prisma schema อธิบาย many-to-many Consumption และ transaction ตอน Confirm
5. เปิด auth flow อธิบาย same-origin proxy, HTTP-only cookies และ refresh rotation
6. เปิด GitHub Actions แสดง lint, typecheck, unit, build, integration และ E2E
7. จบด้วย Gap Analysis เพื่อแสดงว่ารู้ขอบเขตและขั้นตอน production hardening

คำตอบสั้นเมื่อถูกถามว่าอะไรคือส่วนที่ยากที่สุด:

ส่วนที่สำคัญที่สุดคือทำให้ยอดที่หารแล้วรวมกลับมาเท่ากับยอดบิลในทุกกรณี ผมจึงเปลี่ยนการคำนวณเป็น integer cents และแจกเศษตามลำดับที่กำหนด พร้อมทดสอบทั้ง unit และ API integration นอกจากนี้ยังออกแบบ callback และ API proxy ให้ cookie authentication ทำงานได้ในโครงสร้างที่ Frontend กับ Backend แยก service

19. Evidence และสถานะล่าสุด

สถานะที่ยืนยันแล้วใน commit ปัจจุบัน:

Area	ผล
Backend ESLint	ผ่าน
Backend TypeScript	ผ่าน
Backend unit tests	4 ผ่าน
Backend build	ผ่าน
Prisma migration บน CI PostgreSQL	ผ่าน
API integration smoke test	ผ่าน
Frontend ESLint	ผ่าน
Frontend TypeScript	ผ่าน
Frontend unit/component tests	5 ผ่าน
Frontend production build	ผ่าน
Playwright E2E	3 ผ่าน
Archived prototype build	ผ่าน
Dependency audit ล่าสุด	ไม่พบ vulnerability ในทั้ง 3 package trees
README	ไม่ถูกแก้ไขในรอบ portfolio

หลักฐานสำคัญใน repository:

- `.github/workflows/ci.yml`
- `backend/src/party/party.service.spec.ts`
- `backend/scripts/smoke-test.mjs`
- `frontend/test/`
- `frontend/e2e/portfolio.spec.ts`
- `frontend/app/demo/page.tsx`
- `PORTFOLIO.md`

20. Final Assessment

ในมุมมอง portfolio โครงการนี้อยู่ในระดับที่สามารถยื่นสมัครฝึกงานได้ เพราะมี product flow ที่เข้าใจง่าย สถาปัตยกรรม Full Stack จริง business logic ที่มีเหตุผล การจัดการฐานข้อมูลและ authentication รวมถึงหลักฐานด้าน automated testing และ CI

สิ่งที่ควรพูดอย่างตรงไปตรงมาคือระบบยังเป็น portfolio-ready มากกว่า public-production-ready ช่องว่างหลักไม่ได้อยู่ที่ build แต่เป็น operational security, persistent file storage, real OAuth E2E, monitoring และฟีเจอร์ settlement การระบุขอบเขตเหล่านี้ไม่ทำให้ผลงานด้อยลง ตรงกันข้ามจะแสดงให้เห็นว่าสามารถประเมินระบบตามความเสี่ยงและวางแผนพัฒนาต่อได้